Hedgehog — Hack Knight build plan

Oct 8, 2026 · @richie

Goal and demo story

Hedgehog shows which real-world events a portfolio depends on, using live Kalshi odds, and says how far to trust each number. It never trades. Every number on screen is either read straight from Kalshi or measured from years of price history, and the app labels which.

The 3-minute demo judges see:

1. Load a 5-stock portfolio. The exposure map shows which events touch which holdings and how much of the portfolio each one reaches.
2. Flip to **By holding**: each stock with its events, strongest link first (Direct, Measured, Related).
3. Open the calendar: the biggest event this week for this portfolio, with its trust score.
4. Open that event: why the trust score is what it is, and the evidence behind each linked holding.
5. Open the move explainer: today's change split into the whole market and individual stocks, plus the Kalshi markets that moved the same day.
6. Open the track record: when Kalshi said 70%, how often it actually happened, from real settled markets.
7. Finish on the spoken daily briefing.

Pitch line: *"Prediction markets tell you the odds. Hedgehog tells you which odds are about your money, and whether to believe them."*

Tracks and how each is used

Target Best Financial Hack plus three sponsor tracks where the sponsor's tool does real work: Tiger Data, Gemini and ElevenLabs. Best Overall and Runner-Up need no extra work.

Track	What it does in Hedgehog	Where judges see it	Priority
Best Financial Hack	Event exposure, trust scores and a checkable track record for a real portfolio	Every screen	Must
Tiger Data	Time-series tables for odds snapshots and prices; queries for odds changes, same-day moves and calibration	Calendar, Moves, Track record	Must
Gemini API	Tags each Kalshi series from a fixed tag list; reads each stock's 10-K risk factors; writes "why the odds moved" with source links	Exposure map, Event page	Must
ElevenLabs	Speaks a 60-second daily briefing built from the calendar and the day's moves	Overview play button, demo finale	Should
GoDaddy Registry	The project's domain	Demo URL	Should (15 minutes)
Hardware lab	Raspberry Pi desk display of this week's biggest event, or a JBL speaker playing the briefing	Judging table	Could

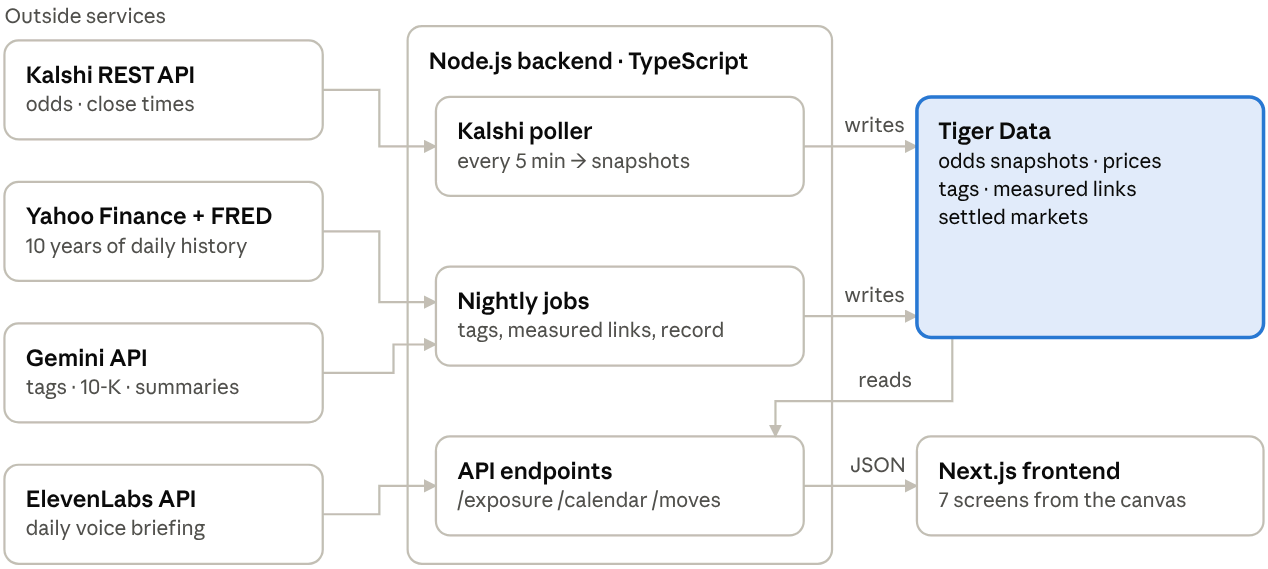
Skip Presage, Health and Environmental. Snowflake overlaps with Gemini, and Solana only fits as an on-chain proof of the track record; add either only if everything above is done.

Open question: how many tracks can one project enter at Hack Knight? Ask an organizer at check-in.

Architecture and data model

A Node.js backend collects and computes; a Next.js frontend only displays; Tiger Data holds everything. Heavy work (tagging, regressions, track record) runs ahead of time, so a page load is a database read.

Node.js collects and computes; the frontend only displays



architecture · 4 outside services, backend, database, frontend

Only the poller and the nightly jobs call outside services, so a slow API never slows a page.

Layer	Choice	Why
Frontend	Next.js + Tailwind	Screens already designed on the canvas; easy deploy
Backend	Node.js + TypeScript (Express)	One language across the app; frontend and backend share each endpoint's types
Scheduled jobs	node-cron inside the backend	Poll Kalshi every 5 minutes; nightly tagging and measurement
Database	Tiger Data (Postgres with time-series tables) through `pg`	Odds and prices are time series; sponsor track
Prices	yahoo-finance2 for stocks and oil; FRED for the 2-year Treasury yield	Free daily history
Math	`hedgehog_math.py` ported to TypeScript, with ml-matrix for the regression	Same formulas and known-answer tests, one runtime

Layer	Choice	Why
AI	Gemini API (Google's JavaScript SDK)	Tagging, 10-K reading, summaries with sources
Voice	ElevenLabs API (its JavaScript SDK)	Daily briefing audio

Tables

Table	Holds	Filled by
`markets`	ticker, series, title, close time, status	Kalshi poller
`odds_snapshots` (time-series)	time, ticker, yes bid, yes ask, midpoint, volume, open interest	Kalshi poller, every 5 minutes
`prices` (time-series)	time, symbol, close	yahoo-finance2 and FRED, daily
`series_tags`	series ticker, tag	Gemini batch, reviewed by you
`stock_tags`	symbol, tag, link type (direct, measured, related)	Your sector table + Gemini 10-K read
`measured_links`	symbol, driver, sensitivity, low, high, days, reliable	Nightly job using `check_sensitivity`
`settled_markets`	ticker, category, odds 24 hours before close, outcome	One-time backfill from Kalshi
`holdings`	session, symbol, dollar value	Add-holdings screen

API endpoints: `POST /holdings`, `GET /exposure`, `GET /calendar`, `GET /event/{ticker}`, `GET /moves?day=`, `GET /earnings/{symbol}`, `GET /track-record`, `GET /briefing`.

Before the hackathon

Everything here is yours to do by hand, and none of it is project code. Most hackathons only allow code written during the event, so check Hack Knight's rules first.

- [ ] Read Hack Knight's rules on what can exist before kickoff (designs and accounts are usually fine; code usually isn't)
- [ ] Confirm the event length and judging time; this plan assumes 24 hours
- [ ] Create a Tiger Data account and a database service; save the connection string
- [ ] Get a Gemini API key from Google AI Studio
- [ ] Create an ElevenLabs account, get an API key, pick a voice
- [ ] Find out how the GoDaddy Registry domain works at the event; shortlist 3 names
- [ ] Pick the demo portfolio: 5 or 6 stocks that have real Kalshi links that week (oil, Fed, CPI, and company markets if any exist). Check on kalshi.com
- [ ] Draft the sector-to-tags table in your notes (about 10 sectors, each with 2 to 5 tags)
- [ ] Install Node 20+, Git and Claude Code; make an empty GitHub repo
- [ ] If you have teammates, split roles: data and backend, frontend, AI and voice plus the pitch

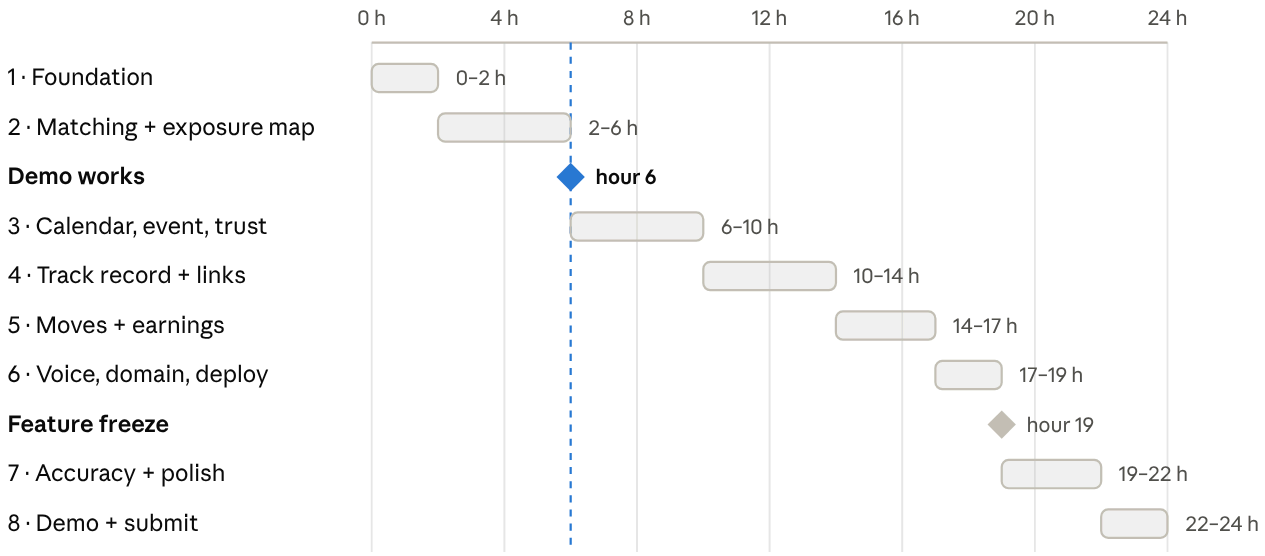
Kalshi needs no account for this app: its public REST endpoints serve market data without a key.

Build phases

Eight phases over 24 hours, each ending in something you can show. The exposure map works by hour 6, so you have a demo even if later phases slip. Each task says who does it: **Agent**, **You**, or **Agent, you review**.

A working demo by hour 6; new features stop at hour 19

Build phases by hour, assuming a 24-hour event



build phases · 8 phases over 24 hours, 2 checkpoints

Phases 1 and 2 give you a demo on their own; everything after adds to it.

1. Foundation (hours 0 to 2). Done when odds rows land every 5 minutes.

- [ ] **Agent:** scaffold the repo (Next.js frontend, Node.js + TypeScript backend with Express, `.env` template)
- [ ] **You:** paste API keys into `.env` yourself; never into chat or the repo
- [ ] **Agent:** create the tables in Tiger Data from the schema above
- [ ] **Agent:** Kalshi poller: open events with their markets every 5 minutes into `markets` and `odds_snapshots`
- [ ] **Agent:** price loader: 10 years of daily closes for the demo stocks, SPY, crude oil futures from yahoo-finance2, plus the 2-year Treasury yield from FRED
- [ ] **You:** compare 3 stored markets against kalshi.com

2. Matching and exposure map (hours 2 to 6). Done when your portfolio shows real events.

- [ ] **Agent, you review:** Gemini batch job tags every Kalshi series from your fixed tag list into `series_tags`
- [ ] **You:** finalize the sector-to-tags table; **Agent** loads it into `stock_tags`
- [ ] **Agent:** `/exposure` endpoint (join on tags, link type, linked value, share of portfolio)

- [ ] **Agent:** add-holdings and exposure-map screens from the canvas designs, including By holding
- [ ] **You:** check every link on screen would make sense to a finance person

3. Calendar, event page and trust score (hours 6 to 10). Done when trust scores are computed live.

- [ ] **You:** confirm the trust weights (spread 30, activity 25, depth 20, track record 25) and bands
- [ ] **Agent:** trust score function, `/calendar` and `/event/{ticker}`
- [ ] **Agent:** Gemini "why the odds moved" grounded in Google Search, so every summary carries links
- [ ] **Agent:** calendar and event screens

4. Track record and measured links (hours 10 to 14). Done when the track record shows real numbers.

- [ ] **Agent:** backfill settled markets for your categories, taking odds 24 hours before close from candlesticks
- [ ] **Agent:** calibration buckets and Brier score per category, `/track-record` and its screen
- [ ] **Agent:** port `hedgehog_math.py` to TypeScript with the same known-answer tests, then run the measured-links job (stocks against oil, the 2-year yield and SPY) with `check_sensitivity`
- [ ] **You:** recompute one reliable and one unreliable result by hand

5. Move explainer and earnings preview (hours 14 to 17).

- [ ] **Agent:** beta split of the day's move, plus same-day odds changes, `/moves` and its screen
- [ ] **You:** check Kalshi lists company markets for your demo stocks; if not, cut the earnings preview
- [ ] **Agent:** earnings preview (normalized revenue brackets, past report-day moves)

6. Voice, domain and deploy (hours 17 to 19).

- [ ] **Agent:** briefing text filled from computed numbers by a template (Gemini may only smooth the wording), sent to ElevenLabs, with a play button
- [ ] **You:** listen to the briefing and check each number it says
- [ ] **You:** register the domain through GoDaddy Registry

- [ ] **Agent:** deploy frontend and backend; connect the domain

7. Accuracy and polish (hours 19 to 22). Run the accuracy checklist below.

8. Demo and submit (hours 22 to 24). Rehearse, record, submit an hour early.

Sleep about 4 hours somewhere in phases 4 or 5, and let a scheduled job run the backfill meanwhile.

Who builds what

Agents can write nearly all of the code. You own the keys, the judgment calls, and every number a judge will hear.

Work	Who	Why
Repo scaffold, tables, endpoints, screens from the canvas	Agent	Well specified and quick to check
Kalshi poller, price loader, settled-market backfill	Agent	Clear inputs and outputs; you spot-check rows
Trust score, beta split, calibration, Brier score	Agent, you review	Easy code, but a silent bug is a wrong number on stage
Gemini tagging, 10-K reading, summaries	Agent, you review	A wrong tag becomes a wrong link on screen
ElevenLabs briefing	Agent, you review	You listen to every number it says
Devpost write-up	Agent drafts, you edit	Your voice and your claims
Accounts, API keys, domain registration	You	Agents shouldn't create accounts or handle your passwords
Sector-to-tags table, trust weights	You (an agent can draft)	Judgment calls judges may question
Demo portfolio	You	Depends on which markets are live that week
Hardware wiring	You	Physical work

Work	Who	Why
Pitch, rehearsal, judging conversations	You	Judges want to hear you explain it

Running agents well:

- Give each agent one area plus its inputs: the tables section of this doc, `hedgehog_math.py`, and the canvas screen it builds.
- Agree the JSON shape of each endpoint first. Then frontend and backend agents can work in parallel, the frontend on mock data.
- Ask for a test with a known answer for every calculation, like the fake-data test already in `hedgehog_math.py`.
- Read every diff that touches money math or tags. Skim the rest.
- Commit at each phase's finish line, so a bad agent run is one revert away.

Accuracy checklist

Run this in phase 7, before anyone outside your team sees the app.

- [ ] No stock move or sensitivity is typed in by hand; every one comes from `measured_links`
- [ ] Any link that fails `check_sensitivity` shows as Related with no number
- [ ] Probabilities are bid-ask midpoints; markets with wide spreads show a Low trust score
- [ ] Bracket ladders are scaled to 100% and say so on screen
- [ ] Track-record odds are taken 24 hours before close, not at close
- [ ] The move explainer adds up: market part plus stock part equals the day's total
- [ ] Every AI summary has source links; a summary without sources is hidden
- [ ] The voice briefing only says numbers from the database
- [ ] "Sample data" labels are gone, and every screen shows when its data was last updated
- [ ] Footer on every screen: not financial advice; odds from Kalshi

Demo script and submission

Three minutes, one portfolio, one story: from "what affects me" to "can I trust it."

Time	Screen	What you say
0:00 to 0:20	Title	Prediction markets price thousands of events, but nobody tells you which ones touch your stocks or how reliable the odds are
0:20 to 0:50	Add holdings, then exposure map	Load the portfolio; point to the event that reaches the most of it; explain Direct, Measured and Related
0:50 to 1:10	By holding	One stock and the events grouped under it, strongest link first
1:10 to 1:40	Calendar, then event page	This week's biggest event for this portfolio, and why its trust score is what it is
1:40 to 2:10	Move explainer	Today's change split into market and stocks; which Kalshi markets moved alongside
2:10 to 2:35	Track record	The calibration headline, from real settled markets
2:35 to 3:00	Briefing audio	Play 15 seconds, then the pitch line and the sponsor tools used

- [ ] Submission page: description, screenshots, and one paragraph per track on how its tool is used
- [ ] Video of the demo (record a rehearsal)
- [ ] Public repo with a README: setup, data sources, the accuracy rules
- [ ] Domain live and pointing at the app
- [ ] Every track you're entering is selected on the form
- [ ] Offline backup: a recorded demo run and a database snapshot, in case venue Wi-Fi fails

Risks, fallbacks and the cut list

The exposure map, calendar and trust score are the floor; never cut them. Freeze new features at hour 19.

Risk	Fallback
Kalshi API slow, rate-limited or down	Pages read stored snapshots; only the poller calls Kalshi
Venue Wi-Fi fails	Recorded demo run plus a local database snapshot
A stock has no reliable measured link	Show it as Related with no number; pick demo stocks with strong drivers (airlines and oil, banks and rates)
Too few settled markets in a category	Show the count and a Low rating rather than hiding the category
Gemini tags something wrong	Fixed tag list, your review, and a manual override table
No Kalshi company markets for the demo stocks	Cut the earnings preview; nothing else depends on it
An agent breaks working code late	Commit at every phase finish line; revert

If you fall behind, cut in this order:

1. Hardware display
2. Earnings preview
3. Voice briefing (costs the ElevenLabs track)
4. Move explainer
5. Track record screen (keep the trust score, using spread, activity and depth only)